Exercise 1: Inventory Management System

Scenario:

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial.

1. Understand the Problem

Why are Data Structures and Algorithms essential?

Data structures organize inventory data efficiently, while algorithms enable fast searching, updating, and deleting of products. They improve performance, reduce processing time, and allow the system to handle thousands of inventory records effectively.

Suitable Data Structures

- **HashMap** – Best for fast searching, updating, and deleting using Product ID. *(Recommended)*
- **ArrayList** – Suitable for storing products sequentially but slower for searching and deletion.
- **LinkedList** – Useful when frequent insertions and deletions are required.
- **TreeMap** – Stores products in sorted order by key.

Welcome | Product.java | Inventory.java | Main.java

Ex1 > Product.java > Product

```java
public class Product {
    private int productId;
    private String productName;
    private int quantity;
    private double price;

    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }

    public int getProductId() {
        return productId;
    }

    public void setQuantity(int quantity) {
        this.quantity = quantity;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    @Override
    public String toString() {
        return "Product ID: " + productId +
               ", Name: " + productName +
               ", Quantity: " + quantity +
               ", Price: " + price;
    }
}
```

Ex1 > Inventory.java > Inventory

```java
import java.util.HashMap;

public class Inventory {

    private HashMap<Integer, Product> products = new HashMap<>();

    // Add Product
    public void addProduct(Product product) {
        products.put(product.getProductId(), product);
        System.out.println(x: "Product Added");
    }

    // Update Product
    public void updateProduct(int id, int quantity, double price) {
        Product product = products.get(id);

        if (product != null) {
            product.setQuantity(quantity);
            product.setPrice(price);
            System.out.println(x: "Product Updated");
        } else {
            System.out.println(x: "Product Not Found");
        }
    }

    // Delete Product
    public void deleteProduct(int id) {
        if (products.remove(id) != null) {
```

```java
Ex1 > J Main.java > Main
public class Main {

    public static void main(String[] args) {

        Inventory inventory = new Inventory();

        inventory.addProduct(new Product(productId: 101, productName: "Laptop", quantity: 20, price: 55000));
        inventory.addProduct(new Product(productId: 102, productName: "Mouse", quantity: 50, price: 500));
        inventory.addProduct(new Product(productId: 103, productName: "Keyboard", quantity: 30, price: 1200));

        inventory.displayProducts();

        inventory.updateProduct(id: 102, quantity: 70, price: 550);

        inventory.deleteProduct(id: 103);

        System.out.println(x: "\nUpdated Inventory:");

        inventory.displayProducts();
    }
```

```
PS C:\Users\anuja\Desktop\Cognizant DSA> cd "c:\Users\anuja\Desktop\Cognizant DSA\Ex1\" ; if ($?) {
Product Added
Product Added
Product Added
Product ID: 101, Name: Laptop, Quantity: 20, Price: 55000.0
Product ID: 102, Name: Mouse, Quantity: 50, Price: 500.0
Product ID: 103, Name: Keyboard, Quantity: 30, Price: 1200.0
Product Updated
Product Deleted

Updated Inventory:
Product ID: 101, Name: Laptop, Quantity: 20, Price: 55000.0
Product ID: 102, Name: Mouse, Quantity: 70, Price: 550.0
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex1>
```

Analysis:

Operation	HashMap Time Complexity
Add Product	O(1) Average
Update Product	O(1) Average
Delete Product	O(1) Average
Search Product	O(1) Average

Exercise 2: E-commerce Platform Search Function

Scenario:

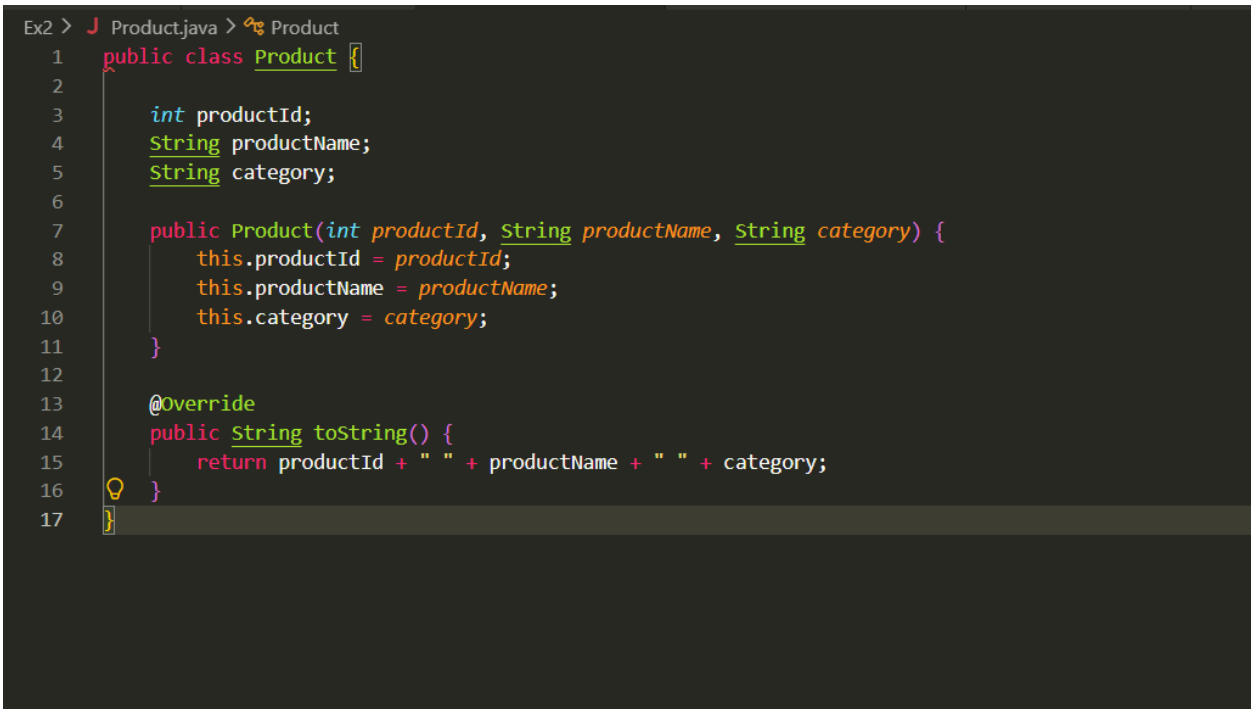
You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Explain Big O Notation

Big O notation measures the time complexity of an algorithm as the input size increases. It helps compare the efficiency of different algorithms and predicts their performance for large datasets.

Best, Average, and Worst Case for Search Operations

- Best Case: The element is found immediately. *(O(1))*
- Average Case: The element is found after checking some of the elements.
- Worst Case: The element is found at the end or is not present. *(O(n) for Linear Search, O(log n) for Binary Search)*

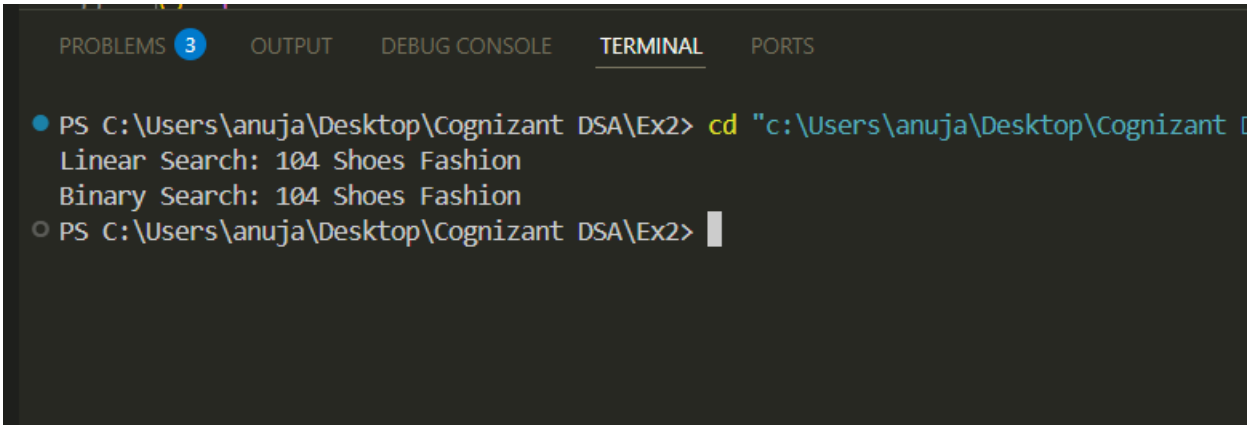


Ex2 > SearchOperations.java > SearchOperations

```java
public class SearchOperations {
    public static Product linearSearch(Product[] products, int id) {
            if (product.productId == id) {
                return product;
            }
        }
        return null;
    }

    // Binary Search
    public static Product binarySearch(Product[] products, int id) {

        int low = 0;
        int high = products.length - 1;

        while (low <= high) {

            int mid = (low + high) / 2;

            if (products[mid].productId == id)
                return products[mid];

            if (products[mid].productId < id)
                low = mid + 1;
            else
                high = mid - 1;
        }

```

Ex2 > Main.java > Main

```java
public class Main {

    public static void main(String[] args) {

        Product[] products = {
                new Product(productId: 101, productName: "Laptop", category: "Electronics"),
                new Product(productId: 102, productName: "Mouse", category: "Electronics"),
                new Product(productId: 103, productName: "Keyboard", category: "Electronics"),
                new Product(productId: 104, productName: "Shoes", category: "Fashion"),
                new Product(productId: 105, productName: "Watch", category: "Accessories")
        };

        Product result1 = SearchOperations.linearSearch(products, id: 104);

        if (result1 != null)
            System.out.println("Linear Search: " + result1);

        Product result2 = SearchOperations.binarySearch(products, id: 104);

        if (result2 != null)
            System.out.println("Binary Search: " + result2);
    }
}
```



Analysis:

Operation	Linear Search	Binary Search
Best Case	**O(1)**	**O(1)**
Average Case	**O(n)**	**O(log n)**
Worst Case	**O(n)**	**O(log n)**

Binary Search is more suitable for an e-commerce platform because it searches products much faster than Linear Search. It has a time complexity of O(log n) but requires the product array to be sorted. Linear Search is simpler but becomes slower as the number of products increases.

Exercise 3: Sorting Customer Orders

Scenario:

You are tasked with sorting customer orders by their total price on an e-commerce platform. This helps in prioritizing high-value orders.

Understand Sorting Algorithms

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. It is simple to implement but inefficient for large datasets.

Insertion Sort

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position. It performs well for small or nearly sorted datasets.

Quick Sort

Quick Sort selects a pivot element and partitions the array into smaller subarrays. It is one of the fastest sorting algorithms for large datasets.

Merge Sort

Merge Sort divides the array into smaller halves, sorts them recursively, and merges them back together. It provides consistent performance but requires extra memory.

Ex3 > Order.java > Order

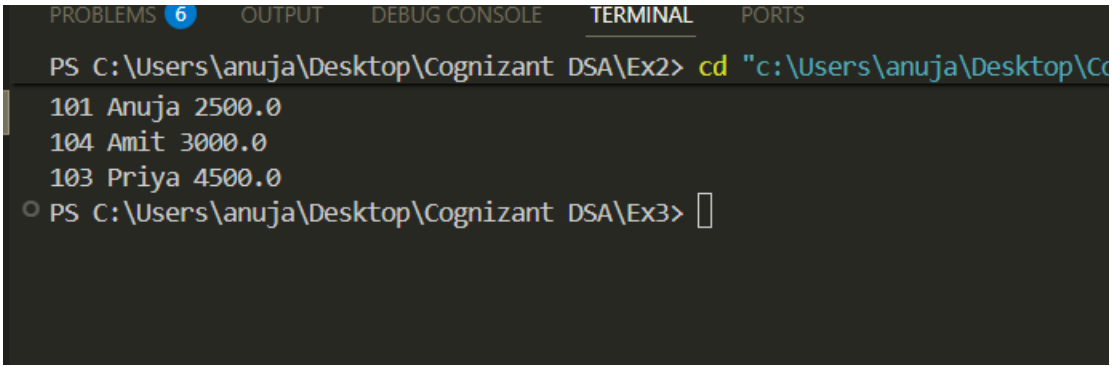
```java
public class Order {

    int orderId;
    String customerName;
    double totalPrice;

    public Order(int orderId, String customerName, double totalPrice) {
        this.orderId = orderId;
        this.customerName = customerName;
        this.totalPrice = totalPrice;
    }

    @Override
    public String toString() {
        return orderId + " " + customerName + " " + totalPrice;
    }
}
```

String customerName

Source: Cognizant DSA_7b889f53

Ex3 > Main.java > Main

```java
public class Main {

    public static void main(String[] args) {

        Order[] orders = {
                new Order(orderId: 101, customerName: "Anuja", totalPrice: 2500),
                new Order(orderId: 102, customerName: "Rahul", totalPrice: 1200),
                new Order(orderId: 103, customerName: "Priya", totalPrice: 4500),
                new Order(orderId: 104, customerName: "Amit", totalPrice: 3000)
        };

        System.out.println(x: "Before Sorting:");
        SortOrders.display(orders);

        SortOrders.quickSort(orders, low: 0, orders.length - 1);

        System.out.println(x: "\nAfter Quick Sort:");
        SortOrders.display(orders);
    }
```



Analysis:

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	**O(n)**	**O(n^2)**	**O(n^2)**
Quick Sort	**O(n log n)**	**O(n log n)**	**O(n^2)**

Why is Quick Sort generally preferred over Bubble Sort?

Quick Sort is generally preferred because it sorts large datasets much faster, with an average time complexity of O(n log n). Bubble Sort has a time complexity of O(n^2), making it inefficient for large numbers of customer orders.

Exercise 4: Employee Management System

Scenario:

You are developing an employee management system for a company. Efficiently managing employee records is crucial.

Explain Array Representation

An array stores elements of the same data type in contiguous memory locations. Each element can be accessed directly using its index, making retrieval fast and efficient.

Advantages of Arrays

- Fast access to elements using an index (O(1)).
- Simple to implement and memory efficient.
- Suitable when the number of elements is fixed.

Ex4 > EmployeeManagement.java > EmployeeManagement

```java
public class EmployeeManagement {

    Employee[] employees = new Employee[10];
    int count = 0;

    // Add Employee
    public void addEmployee(Employee employee) {

        if (count < employees.length) {
            employees[count] = employee;
            count++;
            System.out.println(x: "Employee Added");
        } else {
            System.out.println(x: "Array is Full");
        }
    }

    // Search Employee
    public Employee searchEmployee(int id) {

        for (int i = 0; i < count; i++) {

            if (employees[i].employeeId == id) {
                return employees[i];
            }
        }

        return null;
    }

```

Ex4 > Employee.java > Employee

```java
public class Employee {

    int employeeId;
    String name;
    String position;
    double salary;

    public Employee(int employeeId, String name, String position, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.position = position;
        this.salary = salary;
    }

    @Override
    public String toString() {
        return employeeId + " " + name + " " + position + " " + salary;
    }
}
```

Ex4 > Main.java > Main

```java
public class Main {
    public static void main(String[] args) {
        management.displayEmployees();

        System.out.println(x: "\nSearch Result:");
        System.out.println(management.searchEmployee(id: 102));

        management.deleteEmployee(id: 101);

        System.out.println(x: "\nAfter Deletion:");
        management.displayEmployees();
    }
}
```

PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex3> cd "c:\Users\anuja\Desktop\Cognizant DSA\Ex4\" ; if ($?) { j
Employee Added
Employee Added
Employee Records:
101 Anuja Developer 50000.0
102 Rahul Tester 40000.0
103 Priya Manager 70000.0

Search Result:
102 Rahul Tester 40000.0
Employee Deleted

After Deletion:
102 Rahul Tester 40000.0
103 Priya Manager 70000.0
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex4>
```

Operation	Time Complexity
Add Employee	O(1)
Search Employee	O(n)
Traverse Employees	O(n)
Delete Employee	O(n)

Limitations of Arrays and When to Use Them

Arrays have a fixed size, so they cannot grow dynamically. Insertion and deletion are slow because elements need to be shifted. Arrays are best used when the number of records is known in advance and fast index-based access is required.

Exercise 5: Task Management System

Scenario:

You are developing a task management system where tasks need to be added, deleted, and traversed efficiently.

Understand Linked Lists

Singly Linked List

A Singly Linked List consists of nodes where each node stores data and a reference to the next node. It allows efficient insertion and deletion without shifting elements.

Doubly Linked List

A Doubly Linked List contains two references in each node: one to the next node and one to the previous node. It supports traversal in both forward and backward directions.


Ex5 > TaskLinkedList.java > TaskLinkedList > addTask(Task)
1
2 public class TaskLinkedList {
3
4 private Task head = null;
5
6 // Add Task
7 public void addTask(Task task) {
8
9 if (head == null) {
10 head = task;
11 return;
12 }
13
14 Task temp = head;
15
16 while (temp.next != null) {
17 temp = temp.next;
18 }
19
20 temp.next = task;
21 }

Ex5 > Main.java > Task

```java
public class Main {

    public static void main(String[] args) {

        TaskLinkedList taskList = new TaskLinkedList();

        taskList.addTask(new Task(taskId: 101, taskName: "Design UI", status: "Pending"));
        taskList.addTask(new Task(taskId: 102, taskName: "Write Code", status: "In Progress"));
        taskList.addTask(new Task(taskId: 103, taskName: "Testing", status: "Pending"));

        System.out.println(x: "Task List:");
        taskList.displayTasks();

        System.out.println(x: "\nSearch Result:");
        System.out.println(taskList.searchTask(102));

        taskList.deleteTask(101);

        System.out.println(x: "\nAfter Deletion:");
        taskList.displayTasks();
    }
}
```

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex5> cd "c:\Users\anuja\Desktop\Cogn
Task List:
101 Design UI Pending
102 Write Code In Progress
103 Testing Pending

Search Result:
102 Write Code In Progress

After Deletion:
102 Write Code In Progress
103 Testing Pending
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex5>
```

Analysis:

Operation	**Time Complexity**
Add Task	**O(n)**
Search Task	**O(n)**
Traverse Tasks	**O(n)**
Delete Task	**O(n)**

Advantages of Linked Lists over Arrays

Linked lists are dynamic in size, so memory is allocated as needed. They allow efficient insertion and deletion without shifting elements, making them more suitable than arrays for applications where data changes frequently.

Exercise 6: Library Management System

Scenario:

You are developing a library management system where users can search for books by title or author.

Understand Search Algorithms

Linear Search

Linear Search checks each element one by one until the required item is found or the list ends. It is simple to implement and works on both sorted and unsorted data.

Binary Search

Binary Search repeatedly divides a sorted array into two halves to find the required element. It is much faster than Linear Search but requires the data to be sorted.

```java
class Book {

    int bookId;
    String title;
    String author;

    public Book(int bookId, String title, String author) {
        this.bookId = bookId;
        this.title = title;
        this.author = author;
    }

    @Override
    public String toString() {
        return bookId + " " + title + " " + author;
    }
}

class SearchBook {

    // Linear Search by Title
    public static Book linearSearch(Book[] books, String title) {
```

```java
public class Main {

    public static void main(String[] args) {

        // Array must be sorted by title for Binary Search
        Book[] books = {
                new Book(bookId: 101, title: "Algorithms", author: "Thomas"),
                new Book(bookId: 102, title: "Data Structures", author: "Mark"),
                new Book(bookId: 103, title: "Java Programming", author: "James"),
                new Book(bookId: 104, title: "Operating Systems", author: "Galvin"),
                new Book(bookId: 105, title: "Python Basics", author: "Guido")
        };

        Book book1 = SearchBook.linearSearch(books, title: "Java Programming");

        if (book1 != null)
            System.out.println("Linear Search: " + book1);

        Book book2 = SearchBook.binarySearch(books, title: "Java Programming");

        if (book2 != null)
```

```
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex5> cd "c:\Users\anuja\Desktop\Cognizant DSA\Ex6\" ; if ($?) { javac Mai
Linear Search: 103 Java Programming James
Binary Search: 103 Java Programming James
PS C:\Users\anuja\Desktop\Cognizant DSA\Ex6>
```

Analysis:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	**O(1)**	**O(n)**	**O(n)**
Binary Search	**O(1)**	**O(log n)**	**O(log n)**

When should each algorithm be used?

- Linear Search: Suitable for small or unsorted datasets because it does not require sorting.
- Binary Search: Suitable for large, sorted datasets as it provides much faster search performance with O(log n) time complexity.

Exercise 7: Financial Forecasting

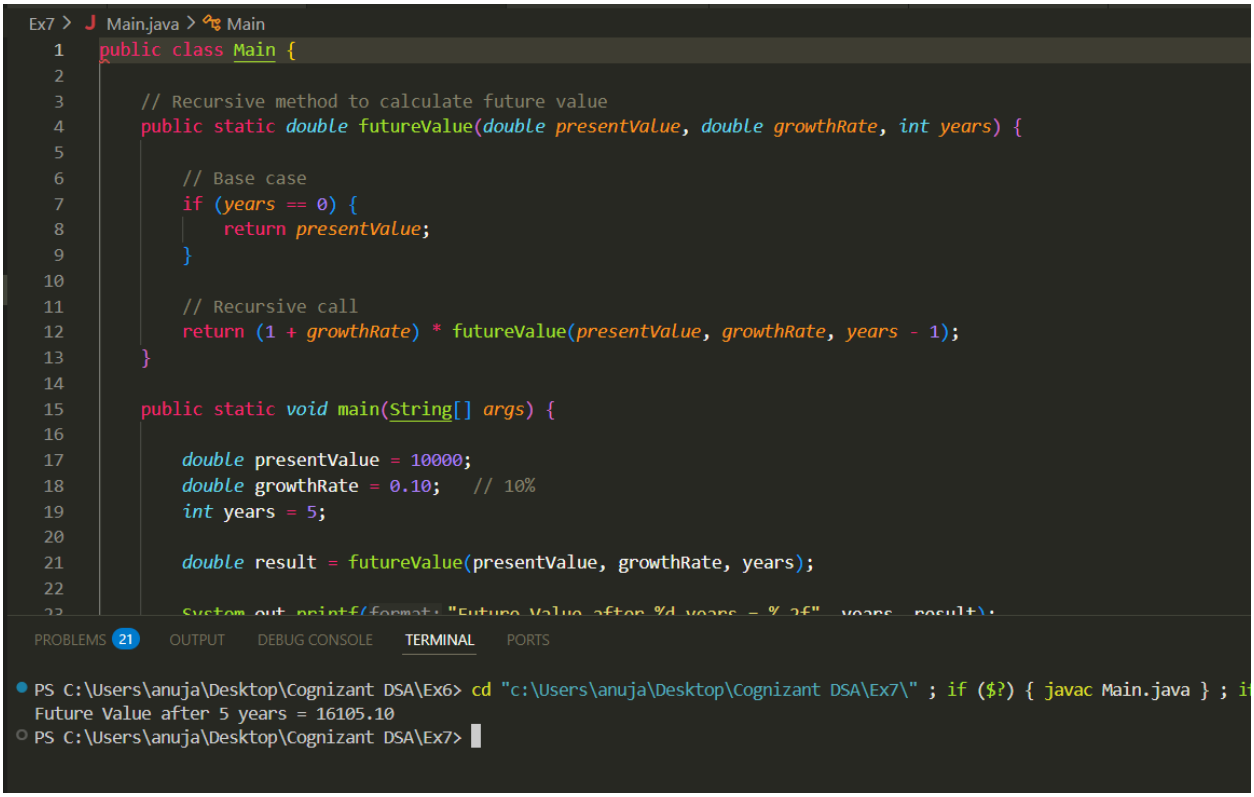
Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Understand Recursive Algorithms

Explain Recursion

Recursion is a programming technique in which a method calls itself to solve a problem. It simplifies problems by breaking them into smaller subproblems until a base case is reached.



Analysis:

Time Complexity

The recursive algorithm makes **one recursive call for each year**, so the time complexity is **O(n)**, where **n** is the number of years. The space complexity is also **O(n)** due to the recursive call stack.

How can the recursive solution be optimized?

The recursive solution can be optimized using **memoization** or **dynamic programming** to avoid repeated computations. For this problem, an **iterative approach** can also be used to eliminate recursion and reduce memory usage.